Universidad Politécnica de Atlautla.

Ing. En tecnologías de la Información e Innovación Digital

Fundamentos de Inteligencia Artificial.

2.5 Algoritmos de Genética.

Docente: Mtro. Javier García Lara

Alumno: Fabian Andrade Hernandez

Grupo: 172

Semana: 26 oct.

Section 5

Instructivo de replicación: AG con DEAP para optimizar una ruta de reparto

Algoritmos genéticos · Proyecto final · Referencia: Wirsansky, caps. 3-5

1. Objetivo y caso de aplicación

Una distribuidora local reparte mercancía a 25 clientes desde un único almacén. El camión debe visitar a cada cliente exactamente una vez y regresar al almacén. Cada kilómetro de más es combustible, tiempo de chofer y entregas que no se hacen. El problema es una instancia del **Problema del Agente Viajero (TSP)**, que es NP-difícil: con 25 clientes existen 24! ≈ 6.2 × 10^{23} rutas posibles, imposibles de revisar una por una.

Un algoritmo genético (AG) explora ese espacio con una población de rutas candidatas que se cruzan, mutan y compiten, de modo que las mejores sobreviven.

Resultado esperado al seguir este instructivo (semilla 42): una ruta de unos 97.8 km, frente a 107.5 km de la heurística del vecino más cercano y 269 km de una ruta aleatoria promedio.

2. Requisitos

- Python 3.9 o superior.
- Librerías: `deap`, `numpy`, `matplotlib`.
- Un editor o terminal. No se necesita GPU ni conexión después de instalar.

3. Paso 1: preparar el entorno

Crea una carpeta de trabajo y, si lo deseas, un entorno virtual:

```
mkdir proyecto_ag && cd proyecto_ag
python -m venv venv
source venv/bin/activate        # en Windows: venv\Scripts\activate
pip install deap numpy matplotlib
```

Verifica la instalación con `python -c "import deap; print(deap.__version__)"`. Debe imprimir un número de versión (por ejemplo, 1.4).

4. Paso 2: definir el problema y los datos

Crea el archivo `ag_ruta_reparto.py`. Primero van los parámetros y los datos del problema. Las coordenadas son sintéticas (zona de 20 × 20 km, almacén al centro); con datos reales bastaría reemplazar la matriz `coords`.

```
SEMILLA = 42
N_CLIENTES = 25
TAM_POBLACION = 200
N_GENERACIONES = 30
P_CRUCE = 0.9
P_MUTACION = 0.4
```

```
TAM_TORNEO = 3
N_ELITE = 2
FRAC_SEMBRADA = 0.25

random.seed(SEMILLA); np.random.seed(SEMILLA)

coords = np.random.uniform(0, 20, size=(N_CLIENTES + 1, 2))
coords[0] = (10, 10)    # fila 0 = almacén
dist = np.linalg.norm(coords[:, None, :] - coords[None, :, :], axis=2)
```

La matriz dist guarda la distancia entre cada par de puntos, de modo que evaluar una ruta es solo sumar entradas. La semilla fija hace que cualquier persona obtenga exactamente los mismos resultados.

5. Paso 3: representar el cromosoma y el fitness

Cromosoma. Una permutación de los clientes: la lista [3, 0, 2, 1, ...] es el orden de visita. Cada cliente aparece una sola vez, por lo que toda ruta es válida.

Fitness. La distancia total almacén → clientes en orden → almacén. Como se busca el mínimo, el peso del fitness es negativo.

```
def longitud_ruta(ruta):
    camino = [0] + [g + 1 for g in ruta] + [0]
    return float(sum(dist[camino[i], camino[i+1]] for i in range(len(camino)-1)))

creator.create('FitnessMin', base.Fitness, weights=(-1.0,))
creator.create('Individuo', list, fitness=creator.FitnessMin)
```

Nota importante: tools.cxOrdered de DEAP solo funciona con permutaciones de 0..n-1. Por eso el gen g representa al cliente g+1 (la fila 0 de dist es el almacén). Si el cromosoma usara los números 1..n, el cruce falla con IndexError: list index out of range.

6. Paso 4: registrar los operadores en el toolbox

```
toolbox = base.Toolbox()
toolbox.register('permutacion', random.sample, range(N_CLIENTES), N_CLIENTES)
toolbox.register('individuo', tools.initIterate, creator.Individuo, toolbox.permutacion)
toolbox.register('poblacion', tools.initRepeat, list, toolbox.individuo)

toolbox.register('evaluate', lambda ind: (longitud_ruta(ind),))
toolbox.register('select', tools.selTournament, tournsize=TAM_TORNEO)
toolbox.register('mate', tools.cxOrdered)

def mut_inversion(individuo):
    i, j = sorted(random.sample(range(len(individuo)), 2))
    individuo[i:j+1] = reversed(individuo[i:j+1])
    return (individuo,)
```

```
toolbox.register('mutate', mut_inversion)
```

Por qué cada operador:

- **Selección por torneo (tamaño 3):** elige 3 individuos al azar y se queda con el mejor. Da presión selectiva sin perder diversidad.
- **Cruce ordenado (cxOrdered):** copia un tramo de un padre y completa con los genes restantes en el orden del otro. Los hijos siempre son permutaciones válidas.
- **Mutación por inversión:** invierte un tramo de la ruta. Equivale a un movimiento 2-opt y elimina cruces entre aristas, que son justo lo que empeora una ruta. Fue el cambio que permitió superar a la heurística voraz; con `mutShuffleIndexes` el AG quedó en 109.4 km en 30 generaciones.

7. Paso 5: crear la población inicial

La población inicial tiene 200 individuos: el 75 % son rutas aleatorias y el 25 % se deriva de la ruta del vecino más cercano (la ruta exacta más variantes con 1 a 3 inversiones). Así el AG parte de una buena solución sin perder diversidad.

```
def vecino_mas_cercano():
    no_visitados, actual, ruta = set(range(1, N_CLIENTES + 1)), 0, []
    while no_visitados:
        sig = min(no_visitados, key=lambda j: dist[actual, j])
        ruta.append(sig - 1)
        no_visitados.remove(sig)
        actual = sig
    return ruta

n_sem = int(TAM_POBLACION * FRAC_SEMBRADA)
pob = toolbox.poblacion(n=TAM_POBLACION - n_sem)
base_voraz = vecino_mas_cercano()
for k in range(n_sem):
    ind = creator.Individuo(base_voraz)
    if k > 0:
        for _ in range(random.randint(1, 3)):
            toolbox.mutate(ind)
    pob.append(ind)
```

8. Paso 6: ciclo evolutivo (30 generaciones)

Cada generación repite: seleccionar, cruzar, mutar, evaluar y aplicar elitismo (los 2 mejores pasan intactos, así el mejor resultado nunca empeora).

```
salon_fama = tools.HallOfFame(N_ELITE)
for ind in pob:
    ind.fitness.values = toolbox.evaluate(ind)
salon_fama.update(pob)
```

```
for g in range(1, N_GENERACIONES + 1):
    descendencia = [toolbox.clone(i) for i in toolbox.select(pob, len(pob) - N_ELITE)]
    for h1, h2 in zip(descendencia[::2], descendencia[1::2]):
        if random.random() < P_CRUCE:
            toolbox.mate(h1, h2)
            del h1.fitness.values, h2.fitness.values
    for m in descendencia:
        if random.random() < P_MUTACION:
            toolbox.mutate(m)
            del m.fitness.values
    for ind in [i for i in descendencia if not i.fitness.valid]:
        ind.fitness.values = toolbox.evaluate(ind)
    pob = descendencia + [toolbox.clone(e) for e in salon_fama]
    salon_fama.update(pob)
    # aquí se registran mejor, promedio y peor fitness de la generación
```

El archivo completo `ag_ruta_reparto.py` incluye este ciclo envuelto en la función `ejecutar()`, con el registro de estadísticas (`tools.Statistics`) por generación.

9. Paso 7: ejecutar

```
python ag_ruta_reparto.py
```

Se imprime una línea por generación (mejor, promedio y peor distancia) y al final un resumen comparativo. Se generan tres imágenes en la misma carpeta: `convergencia.png`, `ruta_voraz.png` y `ruta_ag.png`.

10. Paso 8: interpretar los resultados

Resultados con `SEMILLA = 42`:

Método	Distancia total
Ruta aleatoria (promedio de 1000)	≈ 269 km
Vecino más cercano (voraz)	107.45 km
Algoritmo genético (30 generaciones)	97.79 km

El AG mejora 63.7 % frente a la ruta aleatoria y 9.0 % frente a la heurística voraz.

Gráfica de convergencia (`convergencia.png`). Muestra tres curvas por generación. La del mejor individuo baja de forma escalonada y nunca sube (elitismo). La del promedio sube o se estabiliza por momentos porque la mutación al 40 % introduce rutas peores; es normal y mantiene la exploración. La del peor individuo sirve para ver que la población sigue siendo diversa.

Mapas de ruta (`ruta_ag.png`, `ruta_voraz.png`). La ruta del AG casi no tiene cruces de aristas; la voraz suele terminar con trayectos largos de regreso porque los últimos clientes quedan dispersos.

11. Validación: ¿el resultado depende de la semilla?

Para comprobar que la mejora no es casualidad, se repitió el experimento cambiando `SEMILLA` (lo que también genera un mapa distinto):

Semilla	Voraz (km)	AG (km)
42	107.45	97.79
1	88.35	85.41
7	102.68	98.14
123	114.90	104.65
2024	91.41	86.02
99	107.62	106.08

El AG superó a la heurística en las 6 semillas. En cada una la ganancia fue de 1.4 % a 9.0 %: modesta cuando la ruta voraz ya era buena, mayor cuando era mala. Para un análisis más formal se pueden promediar 30 semillas y reportar media y desviación estándar.

12. Problemas frecuentes

Síntoma	Causa y solución
`IndexError` dentro de `cxOrdered`	El cromosoma usa números 1..n. Debe ser una permutación de 0..n-1.
`ModuleNotFoundError: deap`	Falta instalar o el entorno virtual no está activo.
`creator` advierte que la clase ya existe	Se ejecutó el bloque `creator.create` dos veces en una misma sesión (típico en notebooks). Reinicia el kernel.
Resultados distintos a los de este documento	Cambió `SEMILLA` o el orden de las llamadas a `random`. Usa la semilla 42 y no alteres el orden del código.
Las imágenes no se generan	Verifica permisos de escritura en la carpeta. El script usa el backend `Agg`, que no necesita pantalla.

13. Cómo adaptarlo a un caso real

1. **Datos reales:** reemplaza `coords` por las coordenadas de tus clientes y convierte a kilómetros, o reemplaza `dist` por tiempos o distancias por carretera obtenidos de un servicio de mapas.
2. **Más clientes:** sube `N_CLIENTES` y aumenta `N_GENERACIONES` (con 50 o más clientes, 30 generaciones no bastan).
3. **Restricciones:** para ventanas de entrega o capacidad del camión, agrega a la función de fitness una penalización proporcional al incumplimiento.
4. **Varios camiones:** divide la ruta en tramos dentro del cromosoma o resuelve un TSP por camión tras agrupar clientes.

14. Limitaciones

- El AG es una metaheurística: no garantiza el óptimo, solo una buena solución en tiempo razonable.
- Parte del resultado se debe a la siembra con la heurística voraz. Sin ella, el AG necesita más generaciones para alcanzar la misma calidad.
- Los datos son sintéticos; con tráfico y vías reales la distancia euclidiana es solo una aproximación.

15. Referencias

- Wirsansky, E. *Hands-On Genetic Algorithms with Python*, capítulos 3 a 5.
- Documentación de DEAP: https://deap.readthedocs.io
- Promt; Anthropic. (2026). Claude 5.5. https://claude.ia/ (Prompt: Resolución de actividad y programación de código)